Capstone System Cohesion Check: Requirements to Implementation

Project Title: Code Escape

Educational programming puzzle game (Unity, PC)

Course	Systems Fundamentals
Group Number	IT22
Group Members	Jean Joseph C. Mallari Mark Daniel M. Haguisan Louis Angelo D. Junio Alynus Job C. Tombado
Date prepared	11 October 2026

Implementation status reflects the project files inspected and the Editor tests run on 11 October 2026.

Part I. Core Requirements Alignment

Core Requirement / Capability	Who Needs It?	Why It Matters
The system must present a programming question for the chosen language, check the player's answer against a stored solution, and give a hint and feedback.	Player (student); instructor	This is the learning core of the game. Without answer checking, obstacles cannot unlock and nothing is taught.
The player must be able to move the character, and obstacles and hazards must block or catch the player until the related question is solved.	Player	It links play to learning: every question is triggered by an obstacle.
The system must unlock levels in order, show them on a level map, and allow cleared levels to be replayed.	Player; group (content design)	It enforces the learning order and shows the player where they are in the story.
The system must calculate the player's score (points for solved challenges, a penalty for hints) and show the result.	Player; instructor	Scores give feedback and are the data any later report or ranking would use.
The system must keep the player's progress after the game is closed and restore it when the game is opened again.	Player; group (testing and demonstration)	Sessions are interrupted. Without this, the player would restart the story every time.

Planned capabilities (not counted above). A login and registration system and an administrator account that can change the coding questions are planned and will be implemented later. They are not implemented yet.

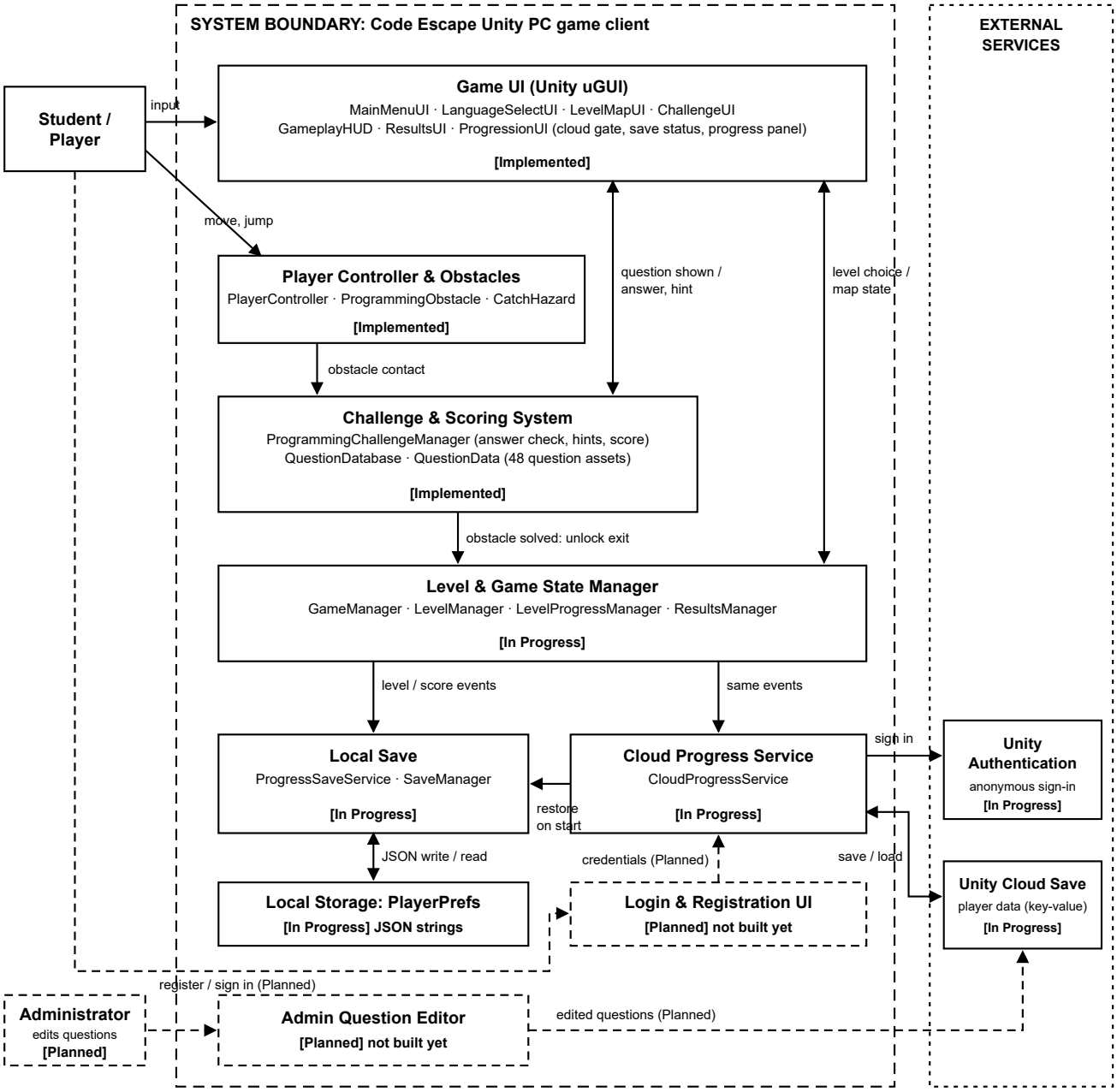
Part II. Requirements-to-Architecture Mapping

Requirement	Responsible Component/s	Data Needed	Current Status
Present and check programming questions	Challenge & Scoring System (`ProgrammingChallengeManager`) + Question Database (`QuestionDatabase`, `QuestionData`) + Game UI (`ChallengeUI`)	question id, language, questionText, choices, correctChoiceIndex, acceptedAnswers, hint, explanation; submitted answer	Implemented (code and 48 question assets; not play-tested)
Move the character; obstacles and hazards gate progress	Player Controller (`PlayerController`) + Obstacles and Hazards (`ProgrammingObstacle`, `CatchHazard`) + Game State (`GameManager`)	movement input, obstacleId, solved flag, GameState, hazard type	Implemented (code and nine obstacles in scenes; not play-tested)
Unlock levels in order, level map, replay	Level & Game State Manager (`LevelManager`, `LevelProgressManager`) + Level Map UI (`LevelMapUI`)	levelId, sceneName, requiredChallenges, unlockedLevels, completedLevels, checkpoints, activeLevel	Implemented (map state confirmed after a restore in the Editor)
Calculate and show score and result	Challenge & Scoring System (score, hints) + `ResultsManager` + Game UI (`GameplayHUD`, `ResultsUI`)	score (+100 / -25), hintsUsed, completed challenges, clues, timePlayed, rank	Implemented (not play-tested)

Requirement	Responsible Component/s	Data Needed	Current Status
Keep and restore progress after the game closes	Local Save (`ProgressSaveService`, `SaveManager`) + Local Storage (PlayerPrefs) + Cloud Progress Service (`CloudProgressService`) + Unity Authentication (anonymous) + Unity Cloud Save	current_level, highest_unlocked_level, completed_levels, total_score, achievements, last_played, language, playthrough state, level progress per language	In Progress. Local save and cloud save are implemented and were tested in the Editor. Login accounts (recoverable identity) are Planned.

Part III. System Architecture

The architecture is the group's Code Escape architecture, updated for this check. It includes only components the project uses or plans to use.



A to I. Architecture elements

Element	In Code Escape
User / Actor	Student / Player (implemented flow). Administrator (planned): changes the coding questions.
Client	Unity game client (Unity 6000.5.10f1), Windows PC
Major user interface	Main menu and level selection, level map, challenge panel and HUD, results, cloud status and progress panel. Planned: Login and Registration UI, Admin Question Editor.

Element	In Code Escape
Core logic	Player Controller, Obstacles and Hazards, Challenge & Scoring System, Level & Game State Manager
Persistent storage	Local: Unity PlayerPrefs (JSON). Cloud: Unity Cloud Save player data (key-value). No SQL or Firebase database.
Backend / API	None of the group's own. The client talks directly to Unity services.
External services	Unity Authentication (anonymous sign-in now), Unity Cloud Save
Data flow	Player input to UI and Player Controller; obstacle contact to the Challenge System; solved obstacle to the Level Manager; level and score events to Local Save and Cloud Progress Service; JSON to PlayerPrefs; sign-in and save/load to Unity services; restored data back to the managers and UI.
System boundary	Dashed rectangle: everything inside is the capstone system. Unity Authentication and Unity Cloud Save are external services.

J. Implementation status

Component	Status
Game UI (menus, level map, challenge, results)	Implemented
Player Controller; Obstacles and Hazards	Implemented
Challenge & Scoring System; Question Database	Implemented
Level & Game State Manager	In Progress
Local Save and Local Storage (PlayerPrefs)	In Progress
Cloud Progress Service, Unity Authentication (anonymous), Unity Cloud Save	In Progress (tested in the Editor only)
Cloud status UI (gate, save status, progress panel)	In Progress (compiles; not yet checked on screen)
Login and Registration UI	Planned
Administrator account and Admin Question Editor	Planned

Architecture explanation

The student interacts with the Unity PC client through the keyboard and mouse: menus, level map, question panel, and character movement. Main processing happens inside the client. Obstacle contact opens a question, `ProgrammingChallengeManager` checks the answer and updates the score, and the level managers unlock levels. Persistent data is stored locally in PlayerPrefs and copied to Unity Cloud Save, which is loaded when the game starts. Components communicate through C# events and singleton references; the client reaches Unity Authentication and Cloud Save through the Unity Gaming Services SDK. The component currently being implemented and verified is the cloud save and its status screen. The next component to implement is the Login and Registration system, because an administrator account and recoverable per-student progress both depend on it.

Part IV. Data and Storage Alignment

The project has no SQL or document database. Existing storage is PlayerPrefs (local) and Unity Cloud Save (cloud).

Data / Entity	Important Fields	Why It Must Persist	Storage
Player progress (cloud record)	player_id, current_level, highest_unlocked_level, completed_levels, total_score, achievements, last_played	Lets the player continue and keeps the score and achievements.	Existing: Unity Cloud Save (`code_escape_progression`)
Restore snapshot	language, progress_json, level_progress per language	Lets the level map, checkpoints and Continue work after a restore.	Existing: Unity Cloud Save (`code_escape_snapshot`)
Playthrough state	storyStage, currentLevel, activeLevel, completedChallenges, completedQuestions, hintedQuestions, collectedClues, score, hintsUsed, timePlayed	Continues the story where the player stopped.	Existing: PlayerPrefs (`CodeEscape.Progress`)
Level map progress	language, completedLevels, lastPlayedLevel, checkpoints	Keeps cleared levels unlocked, separately per language.	Existing: PlayerPrefs (`CodeEscape.LevelProgress.<language>`)
Selected language	language	Decides the question set and level progress.	Existing: PlayerPrefs
Settings	musicVolume, sfxVolume, fullscreen, resolution	Keeps the player's preferences.	Existing: PlayerPrefs
Question bank	id, language, topic, difficulty, questionText, choices, correct answer, hint	Authored content, read-only for players.	Existing: 48 ScriptableObject assets in the project. Planned: an editable store for administrators (not decided).
Student account	player_id, username	Would tie progress to a person.	Planned: Unity Authentication
Administrator role	player_id, role	Would control who may edit questions.	Planned (server-side check needed)

Temporary data (session only): active challenge and selected answer, GameState, current hazard, dialogue and cutscene queue, cloud connection state and save status (`Starting`, `Ready`, `Failed`, `Unsaved`, `Saving`).

Persistent data: everything in the table marked Existing, except the question bank, which is project content rather than player data. Student account, administrator role and an editable question store are Planned.

Part V. Current Implementation Evidence

Evidence	Current Status	Requirement / Component Supported
`Assets/Scripts/Programming/ProgrammingChallengeManager.cs` (points 100, hint penalty 25) and 48 question assets in `Assets/ScriptableObjects/Questions`	Implemented	Questions and score; Challenge & Scoring System
`Assets/Scripts/Player/PlayerController.cs`, `Assets/Scripts/Obstacles/ProgrammingObstacle.cs`, `CatchHazard.cs`; nine obstacle IDs in scenes `03_Home` to `08_FinalArea`	Implemented	Movement and obstacles
`Assets/Scripts/Game/LevelManager.cs`, `LevelProgressManager.cs`, scene `01b_LevelMap.unity`; six levels listed in `GameSystems.prefab`. In the Editor, after a restart the level map state was Home completed, Neighborhood available, Warehouse locked.	Implemented	Level progression
`Assets/Scripts/SaveSystem/SaveManager.cs`, `ProgressSaveService.cs` (PlayerPrefs JSON)	Implemented	Local save; persistent progress
`Assets/Scripts/Cloud/CloudProgressService.cs`, `PlayerProgressData.cs`; packages in `Packages/manifest.json` (Cloud Save 3.4.1). Editor log lines showed a record created, saved after a level clear without duplicates, and loaded on restart.	Implemented (Editor tested)	Persistent progress; Cloud Progress Service, Unity Authentication, Unity Cloud Save
Unity Dashboard, Cloud Save, Player Data (production): a player entry with the keys `code_escape_progression` and `code_escape_snapshot` (screenshot taken by the group member)	Implemented	Cloud storage
`Assets/Scripts/Cloud/ProgressionUI.cs` (cloud gate, status chip, progress panel)	In Progress	Cloud status display; layout not yet checked on screen
GitHub repository `IT22_DatabaseConnectivity` (https://github.com/jjcmall/IT22_DatabaseConnectivity), branch `main`, with a Unity `.gitignore` and `README.md`	In Progress	Repository; project files and commit history are being pushed, no commit hash recorded in this document yet

Evidence the group still needs to add: screenshots of the main menu, level map, challenge panel and results screen; commit hashes; a note of any hand play-test of the questions and score.

Part VI. Next Technical Milestone

Current Gap	The game signs the player in anonymously. Progress belongs to a device and cannot be recovered on another device or after cleared data. There is no login or registration, so progress cannot be tied to a named student, and there is no way to create an administrator account.
Next Technical Milestone	Implement the Login and Registration system (milestone 1 in the Capstone Technical Design and Implementation Baseline): a Login and Registration UI connected to Unity Authentication, replacing anonymous sign-in. The sign-in method will be chosen by the group from those Unity Authentication supports.
Reason	Recoverable per-student progress, and the planned administrator account that edits questions, both depend on a signed-in identity. The cloud save and the sign-in step already exist, so this milestone adds an interface and a sign-in method without adding new technology.
Expected Output	A student can register and sign in; progress is saved and loaded under that account and is available after signing in on another device; sign-in errors are shown with a retry; the architecture diagram shows the Login UI as Implemented.

Part VII. Group Contribution Traceability

The specific contributions and commit hashes below must be completed by the group. Nothing has been assigned on anyone's behalf. The "Related Artifact" column lists the real parts of the project to choose from.

Group Member	Assigned / Actual Contribution	Related Artifact or Component	Evidence
Jean Joseph C. Mallari	*[specific contribution]*	Part I and II: requirements mapping	*[table or page]*
Mark Daniel M. Haguisan	*[specific contribution]*	Part III: architecture diagram	*[diagram section]*
Louis Angelo D. Junio	*[specific contribution]*	Part IV: data and storage design (`Assets/Scripts/SaveSystem`, `Assets/Scripts/Cloud`)	*[file or commit hash]*
Alynus Job C. Tombado	*[specific contribution]*	Gameplay scripts, scenes and UI (`Assets/Scripts`, `Assets/Scenes`)	*[file, scene or commit hash]*

The "Related Artifact" cell next to each name is only a suggestion. Replace it with what that member actually did.

Part VIII. Individual Participation Record

Each student must complete this page personally. It must reflect that student's own work and must not be copied from another member.

Student Name	*[your name]*
Group Number	IT22
Primary Responsibility	*[your main responsibility]*
1. My Actual Contribution	*[the specific task or tasks you completed]*
2. Artifact or Component I Worked On	*[where your work can be found]*
3. Evidence of Contribution	*[commit hash, screenshot, file, page or test result]*
4. Technical Decision or Learning (3 to 5 sentences)	*[one technical decision, issue, correction or learning you personally met]*
5. Next Responsibility	*[the next technical task you are expected to do]*